



Security Review For Kaskad



Collaborative Audit Prepared For:
Lead Security Expert(s):

Kaskad
hildingr
TessKimy

Date Audited:

February 16 - March 9, 2026

Introduction

Kaskad - A bounded governance lending protocol with in-house COB Oracle integration for DeFi ecosystems.

Scope

Repository: Kaskad-Lending/kaskad-squashed

Audited Commit: dcb8881f4238a23e4d4cd71c863a01d1bf608c6f

Final Commit: 28307ddb107874a1309d50897b975cde2c3ee41c

Files:

- src/config/kaskad/EpochConfig.sol
- src/dao/EmissionManager.sol
- src/dao/KaskadActivityTracker.sol
- src/dao/KaskadGovernor.sol
- src/dao/KaskadStrategy.sol
- src/dao/KSKDEmissionVault.sol
- src/interfaces/IEmissionManager.sol
- src/interfaces/IKaskadActivityTracker.sol
- src/interfaces/IKaskadStrategy.sol
- src/KaskadRewardsController.sol
- src/KSKD.sol
- src/pools/GrowthPool.sol
- src/pools/StKSKDVault.sol
- src/revenue-splitter/BasketRevenueSplitter.sol
- src/revenue-splitter/DaoRevenueSplitter.sol
- src/utils/Basket4626.sol
- src/utils/DecisionParams.sol
- src/utils/SupplyAdjustment.sol
- src/vesting/CommunityRoundVesting.sol
- src/vesting/TeamVesting.sol

Final Commit Hash

28307ddb107874a1309d50897b975cde2c3ee41c

Findings

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

Issues Found

High	Medium	Low/Info
9	8	4

Issues Not Fixed and Not Acknowledged

High	Medium	Low/Info
0	0	0

Issue H-1: Incorrect User Index Initialization Using Static Function [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/26>

Summary

The `_initializeUserIndexIfNeeded()` function uses `REWARDS.getRewardIndex()` which returns a static value that may not reflect the correct current index, causing users to miss rewards or receive incorrect reward calculations.

Vulnerability Detail

When initializing user rewards tracking, the function captures the reward index using a getter function:

```
ur.userIndex = uint128(REWARDS.getRewardIndex(asset, reward));

...

function getRewardIndex(
    address asset,
    address reward
) external view returns (uint256) {
    return _assets[asset].rewards[reward].index;
}
```

If `getRewardIndex()` returns a cached or stale value rather than the dynamically calculated current index, the `userIndex` will be incorrect. When rewards are later calculated using the formula $(\text{endIndex} - \text{userIndex})$, users may receive more rewards than entitled.

Impact

If users consistently receive more rewards than entitled due to low starting indices, the reward pool may become insolvent.

Tool Used

Manual Review

Recommendation

Ensure the reward index is properly updated before capturing it for user initialization.

Discussion

KaiReytsu

Fixed. The call ordering in `handleAction()` ensures the user index is captured before the snapshot updates the global index. `_initializeUserIndexIfNeeded()` runs first, recording the current `rewardIndex` via `getRewardIndex()`. Only then does `_maybeSnapshotPreviousEpoch()` update the global index. This guarantees that $(\text{snapshotIndex} - \text{userIndex}) > 0$ when claiming, and the user accrues only rewards earned after their first interaction.

```
// 1. Initialize user index FIRST
_initializeUserIndexIfNeeded(user, asset, currentEpoch);
// 2. Snapshot previous epoch (updates global index)
_maybeSnapshotPreviousEpoch(asset, currentEpoch, totalSupply);
```

DemoreXTess

Index initialization is not correct. `_initializeUserIndexIfNeeded` calls `getRewardIndex` function of rewards controller. This function is always provide stale and cached index.

```
function getRewardIndex(
    address asset,
    address reward
) external view returns (uint256) {
    return _assets[asset].rewards[reward].index;
}
```

When user is eligible to start earning rewards, this index will be used and user will earn more rewards than expected amount.

KaiReytsu

Made the updates. PR: <https://github.com/Kaskad-Lending/kaskad-squashed/pull/14>

Issue H-2: New Rewards Not Distributed to Existing Token Holders [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/27>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

When a new reward token is added to an asset after users have already deposited, those existing users will not receive the new reward because their UserRewards struct is never initialized for the new reward.

Vulnerability Detail

The `_processAssetRewards()` function iterates through available rewards and processes them:

```
for (uint128 r = 0; r < numRewards; ++r) {
    address reward = REWARDS.getAssetReward(ctx.asset, r);
    UserRewards storage ur = _userRewards[ctx.user][ctx.asset][reward];

    // Skip if never initialized or already processed
    if (!ur.initialized || ur.lastClaimedEpoch >= ctx.currentEpoch)
        continue;
```

User reward tracking is only initialized in `_initializeUserIndexIfNeeded()`, which is called during `handleAction()`. If a user deposited before a new reward was added, their UserRewards for that new reward will have `initialized = false`, causing all reward processing to be skipped.

Impact

Existing depositors will never receive newly added reward tokens, even though they are eligible based on their position.

Tool Used

Manual Review

Recommendation

Implement a mechanism to initialize reward tracking for existing users when new rewards are added.

Discussion

KaiReytsu

Acknowledged. This is a standard lazy-initialization pattern, same as Aave's RewardsController. ERC20 provides no `getHolders()`, so mass-initializing all existing users when a new reward is added is not feasible without a holder registry (which would add gas overhead to every transfer). When a user next interacts, `_initializeUserIndexIfNeeded` iterates over all current rewards, including newly added ones, and initializes any that are missing. The user starts accruing the new reward from their first interaction after it is added. No previously earned rewards are lost since `userIndex` is set to the current global index and the delta starts at zero.

Issue H-3: Epoch Snapshot Timing Causes Reward Insolvency [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/28>

Summary

The reward index snapshot mechanism can capture additional rewards beyond what was allocated for an epoch if triggered after the official epoch finalization, leading to protocol insolvency where some users cannot claim their entitled rewards.

Vulnerability Detail

The `_snapshotEpoch()` function calls `updateRewardIndexForTracker()` to capture the current reward index:

```
uint256 index = REWARDS.updateRewardIndexForTracker(  
    asset,  
    reward,  
    actualTotalSupply  
);
```

The `KaskadRewardsController` operates on a reward-rate-per-second basis while official epoch handling works monthly based. If the snapshot is triggered one hour after the official epoch end, the index will include an additional hour's worth of rewards that were not allocated for that epoch.

Impact

The recorded index includes rewards beyond what was distributed for the epoch. When all users try to claim based on this inflated index, the total claims will exceed available tokens.

Tool Used

Manual Review

Recommendation

Implement a time-bounded index calculation that only accounts for rewards up to the official epoch end timestamp.

Discussion

KaiReytsu

Fixed. Fix details in <https://github.com/Kaskad-Lending/kaskad-squashed/pull/5>

Issue H-4: Update to entryTime when calling stake() can circumvented [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/34>

Summary

StKSKDVault::_update() does not dilute the receiver's entryTime when they already have a non-zero entry, allowing users to bypass the weighted-average time dilution enforced by stake() by transferring shares from a fresh wallet.

Vulnerability Detail

When a user calls stake(), the contract applies a weighted-average calculation to move their entryTime forward, reducing their accumulated holdingDuration():

```
// Weighted average entry time
if (oldShares == 0) {
    entryTime[msg.sender] = uint64(block.timestamp);
} else {
    uint64 oldEntry = entryTime[msg.sender];
    uint64 newEntry = uint64(
        (uint256(oldEntry) * oldShares + block.timestamp * shares) /
        (oldShares + shares)
    );
    entryTime[msg.sender] = newEntry;
}
```

However, in _update(), when shares are transferred to a recipient who already has a non-zero entryTime, no dilution is applied:

```
// Only initialize entryTime for new recipients
if (entryTime[to] == 0) {
    entryTime[to] = uint64(block.timestamp);
}
// If entryTime[to] != 0, keep their existing accumulated time
```

This creates an inconsistency that can be exploited:

1. Attacker has a main account with old entryTime (high holdingDuration = high voting power).
2. Attacker stakes new KSKD on a fresh wallet via stake().
3. Fresh wallet transfers all shares to the main account.

4. Main account receives additional shares **without any** entryTime **dilution** – its holdingDuration() remains unchanged.

If the attacker had instead called `stake()` directly on the main account, the weighted-average would have moved entryTime forward, reducing their holdingDuration() and voting power.

Impact

Users can accumulate arbitrary additional stKSKD shares on an account without any reduction in holdingDuration(). This inflates their voting power beyond what the time-weighted mechanism intends to allow, undermining the fairness of any governance or reward system that relies on holdingDuration().

Recommendation

Presumable this was removed before because other users could force updates on any receiver by transferring shares to them.

Removing this opens up this new attack vector where we can completely circumvent the update to the entryTime when increasing our staking position.

Given that the new entryTime is scaled based on the 'amount' transferred an attacker would have to send a significant amount to another user to alter their entryTime. This would be in the interest of the receiver as they receive valuable assets.

If you wish to mitigate the above completely you could allow users to completely block transfers to them by setting a flag in their account.

Discussion

KaiReytsu

Fixed. Fix details in <https://github.com/Kaskad-Lending/kaskad-squashed/pull/7>

Issue H-5: Undistributed rewards are never distributed and are instead locked in the contract. [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/35>

Summary

Recycled undistributed rewards are accounted in `recycledForEpoch` for the next epoch but never actually added to that epoch's reward budget, permanently locking the tokens in the `KaskadRewardsController` contract.

Vulnerability Detail

When ineligible users' rewards are processed, the amounts are recorded as undistributed via `recordUndistributed()`. When an epoch ends, these undistributed rewards are recycled, a portion is burned, and the remainder is earmarked for the next epoch:

<https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/blob/main/kaskad-squashed/src/KaskadRewardsController.sol#L155-L163>

```
epochRecycled[epoch][reward] = true;

uint16 toNextEpochBps = STRATEGY.undistributedToNextEpochBps();
uint256 toNextEpoch = (undistributed * toNextEpochBps) / 10000;
uint256 toBurn = undistributed - toNextEpoch;

if (toNextEpoch > 0) {
    recycledForEpoch[epoch + 1][reward] += toNextEpoch;
}
```

The `recycledForEpoch` mapping is written to but **never consumed** by any distribution logic. The next epoch's emission is pulled exclusively from the vault in `EmissionManager._distributeEpochEmission()`:

```
function _distributeEpochEmission(uint256 epoch) internal {
    if (rewardsController == address(0)) revert ZeroAddress();
    uint256 totalEmission = vault.pullForEpoch(epoch, rewardsController);
    // ...
}
```

The `emissionPerSecond` configured via `configureAssets()` is also derived solely from `vault.epochEmission()` it does not account for recycled amounts. The `KaskadActivityTracker` computes user rewards purely from index growth driven by `emissionPerSecond`, and never reads `recycledForEpoch`.

The only reader of `recycledForEpoch` is the view function `getRecycledForEpoch()`, which has no callers within the protocol.

Impact

The `toNextEpoch` portion of undistributed rewards (determined by `undistributedToNextEpochBps`) is neither burned nor redistributed, it remains locked in the `KaskadRewardsController` contract.

Recommendation

We need to account for the recycled rewards in the next epoch's reward budget by increasing the `emissionPerSecond` for the next epoch such that it reflects the recycled amount.

Discussion

KaiReytsu

Fixed. Fix details in <https://github.com/Kaskad-Lending/kaskad-squashed/pull/8>

hildingr

The `_maybeDistributePreviousEpoch()` flow at `KaskadRewardsController.sol:152-165` calls `_autoRecycleEpoch(N)` then `_updateEmissionRate(N+1)` but `closeUserEpochs` runs after both (lines 463-466). So `_updateEmissionRate` sets `rolloverApplied[N+1] = true` on the second claim (line 206), and all undistributed generated by subsequent claimers gets recycled into `recycledForEpoch[N+1]` (line 184) but never applied to emissions because the boolean blocks re-entry at line 205.

Recycled rewards are stranded in `recycledForEpoch` with no mechanism to apply or recover them.

KaiReytsu

Made the updates. PR: <https://github.com/Kaskad-Lending/kaskad-squashed/pull/16>

Issue H-6: Incorrect epoch is used in voting weight calculation, TVL component will be nullified. [RE-SOLVED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/36>

Summary

`_calculateTvlComponent()` uses `governor.currentEpoch()` (epoch N) to compute uptime, but voting in `KaskadGovernor.vote()` occurs during epoch N for epoch N-1. This means voting weight is calculated against the just-started current epoch rather than the completed epoch being voted on, resulting in artificially suppressed and time-dependent voting weights.

Vulnerability Detail

When a user calls `vote()`, the governor determines the voting epoch as `current - 1`:

<https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/blob/main/kaskad-squashed/src/dao/KaskadGovernor.sol#L111-L114>

```
uint256 current = currentEpoch();
if (current == 0) revert NoPastEpoch();

uint256 epoch = current - 1;
```

Voting only opens after epoch N-1 ends (enforced at line 122: `if (block.timestamp < epochEnd) revert VotingWindowNotStarted()`), so voters are always in epoch N when casting votes for epoch N-1.

The governor then fetches the user's voting weight via the strategy:

<https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/blob/main/kaskad-squashed/src/dao/KaskadGovernor.sol#L141-L143>

```
strategy.syncForVoting(msg.sender);

uint256 weight = strategy.votingWeight(msg.sender);
```

Inside `_calculateTvlComponent()`, the epoch used for uptime calculation is `governor.currentEpoch()`, which returns epoch N (the current, in-progress epoch):

```
function _calculateTvlComponent(
    address account
) internal view returns (uint256) {
    uint256 epoch = governor.currentEpoch();
```

```

    UserEpochState storage state = userEpochState[account];

    uint256 supplyTerm = _calculateSupplyTerm(state, epoch);
    uint256 borrowTerm = _calculateBorrowTerm(account, state, epoch);

    return (lambdaTvl * (supplyTerm + borrowTerm)) / PRECISION;
}

```

Both `_calculateSupplyTerm()` and `_calculateBorrowTerm()` pass this epoch to `_calculateUptimeBps()`, which computes uptime as the fraction of the epoch the user has been eligible:

```

uint256 effectiveEnd = block.timestamp < epochEnd
    ? block.timestamp
    : epochEnd;

// ...

uint256 eligibleTime = effectiveEnd - effectiveStart;
return uint16((eligibleTime * 10_000) / epochDuration);

```

Since voting occurs right after epoch N-1 ends (i.e., at the start of epoch N), `block.timestamp` is close to `epochStart` of epoch N, resulting in a near-zero uptime value.

Critically, `_calculateSupplyTerm()` and `_calculateBorrowTerm()` enforce minimum uptime thresholds before returning a non-zero value:

```

uint16 public supplierUptimeThresholdBps = 9000; // 90%
uint16 public borrowerUptimeThresholdBps = 5500; // 55%

```

At [line 250](#) and [line 287](#):

```

if (uptimeBps < supplierUptimeThresholdBps) {
    return 0;
}

```

The voting window is 7 days (`VOTING_WINDOW = 7 days`) and the epoch duration is 30 days (`DURATION = 30 days`). Even at the very end of the voting window, the maximum possible uptime against epoch N is $7 / 30 = 23.3\%$, far below both the 55% borrower threshold and the 90% supplier threshold. Both `_calculateSupplyTerm()` and `_calculateBorrowTerm()` will **always return 0** during the entire voting window, making the TVL component permanently zero for all voters.

Impact

The TVL component of voting weight is **completely nullified** for all users across the entire voting window. Since the maximum achievable uptime against the current epoch

(≈23.3%) can never meet the minimum thresholds (55%/90%), both supply and borrow terms always return 0. This means:

- Voting weight is reduced to only the loyalty term (staking + activity), entirely removing the TVL-based governance influence that the protocol explicitly designed.
- Users with large supply/borrow positions but modest staking positions lose their intended governance power.
- If the loyalty term is also zero for a user (e.g., no stKSKD staked), the `vote()` call reverts with `ZeroWeight()` at line 146, completely preventing them from voting.

Recommendation

Use the previous (completed) epoch for uptime calculation within `_calculateTvlComponent()`, aligning the uptime measurement with the epoch being voted on:

```
function _calculateTvlComponent(
    address account
) internal view returns (uint256) {
-   uint256 epoch = governor.currentEpoch();
+   uint256 currentEp = governor.currentEpoch();
+   uint256 epoch = currentEp > 0 ? currentEp - 1 : 0;
    UserEpochState storage state = userEpochState[account];

    uint256 supplyTerm = _calculateSupplyTerm(state, epoch);
    uint256 borrowTerm = _calculateBorrowTerm(account, state, epoch);

    return (lambdaTvl * (supplyTerm + borrowTerm)) / PRECISION;
}
```

Discussion

KaiReytsu

Fixed. Fix details in <https://github.com/Kaskad-Lending/kaskad-squashed/pull/9>

Issue H-7: Attacker can zero out users tvlComponent by sending a dust amount [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/37>

Summary

An attacker can permanently zero out any targeted user's `tvlComponent` voting weight by front-running their `vote()` transaction with a dust `AToken` transfer. This triggers `handleAction() → syncParticipation() → _snapshotPositions()`, which resets `lastSyncTimestamp` to `block.timestamp`, causing the 1-hour `VOTING_COOLDOWN` check in `_calculateSupplyTerm()` and `_calculateBorrowTerm()` to return 0. Since the attacker can repeat this on every vote attempt, the victim's TVL-based governance weight is effectively suppressed indefinitely.

Vulnerability Detail

When a user calls `vote()`, the governor calls `syncForVoting()` which was specifically designed to **not** update `lastSyncTimestamp`, then reads the voting weight:

<https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/blob/main/kaskad-squashed/src/dao/KaskadGovernor.sol#L141-L143>

```
strategy.syncForVoting(msg.sender);

uint256 weight = strategy.votingWeight(msg.sender);
```

The `VOTING_COOLDOWN` mechanism is intended to prevent flash loan attacks – if `lastSyncTimestamp` was set within the last hour, both `_calculateSupplyTerm()` and `_calculateBorrowTerm()` return 0:

<https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/blob/main/kaskad-squashed/src/dao/KaskadStrategy.sol#L235-L237>

```
if (state.lastSyncTimestamp + VOTING_COOLDOWN > block.timestamp) {
    return 0;
}
```

<https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/blob/main/kaskad-squashed/src/dao/KaskadStrategy.sol#L265-L267>

```
if (state.lastSyncTimestamp + VOTING_COOLDOWN > block.timestamp) {
    return 0;
}
```

However, there is a separate path that updates `lastSyncTimestamp` which an attacker can trigger externally. When any AToken balance changes (including transfers), the AToken contract calls `KaskadRewardsController.handleAction()`:

```
function handleAction(
    address user,
    uint256 totalSupply,
    uint256 userBalance
) external override {
    if (address(activityTracker) != address(0)) {
        activityTracker.handleAction(
            msg.sender,
            user,
            totalSupply,
            userBalance
        );
    }
}
```

This flows through `KaskadActivityTracker.handleAction()` which calls `STRATEGY.syncParticipation(user)` at line 144.

`syncParticipation()` calls `_snapshotPositions()`, which unconditionally resets the cooldown timestamp:

<https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/blob/main/kaskad-squashed/src/dao/KaskadStrategy.sol#L495-L496>

```
// Record sync timestamp for flash loan protection
state.lastSyncTimestamp = uint64(block.timestamp);
```

The attack:

1. Attacker monitors the mempool for the victim's `vote()` transaction
2. Attacker front-runs by transferring a dust amount of ATokens to the victim (e.g. 1 wei)
3. The AToken transfer triggers `handleAction()` → `syncParticipation()` → `_snapshotPositions()`, resetting `state.lastSyncTimestamp = block.timestamp`
4. The victim's `vote()` executes in the same block. `syncForVoting()` does not overwrite `lastSyncTimestamp`, so the cooldown check `lastSyncTimestamp + VOTING_COOLDOWN > block.timestamp` evaluates to **true**
5. Both `_calculateSupplyTerm()` and `_calculateBorrowTerm()` return 0, zeroing the `tv1` Component
6. The attacker can repeat this on every vote attempt at negligible cost (dust transfer gas)

Note that while `syncForVoting()` was correctly designed to avoid resetting `lastSyncTimestamp`, this protection is bypassed entirely because the attacker triggers the `syncParticipation()` path externally via a dust AToken transfer.

Impact

Any user's `tv1Component` can be zeroed out at will by an attacker front-running their vote with a dust AToken transfer. Since the attacker can repeat this every time the victim attempts to vote, the suppression is effectively permanent:

- Users with significant supply/borrow positions lose their TVL-based governance influence entirely.
- If the victim has no loyalty term (no `stKSKD` staked), `votingWeight()` returns 0 and `vote()` reverts with `ZeroWeight()`, completely disenfranchising them from governance.
- The attack cost is negligible (dust AToken transfer gas per vote attempt), while the impact is the ability to selectively silence any voter's TVL-weighted governance power.

Recommendation

The fix is not straightforward because we want to block flash-loans but we do not want transfers from other users to zero out our voting. One potential solution is to only reset the sync timer when the transfer amount is $> x\%$ of the user's total position. Dust amount will not be able to zero-out users TVL. We still block large flashloan attacks as they would have to be significant to have an impact.

We would have to accept that a $x\%$ transfer to a user could reset their TVL component. We also have to accept that user can manipulate their TVL component by a small amount with a flashloan under $x\%$.

Alternatively redesign the system with a commit phase such that any increase in their position after they commit is ignored and require that the commit is $\Rightarrow$ 1h ago, we check that the current balance is not lower than the balance when we commit.

Discussion

KaiReytsu

Fixed. Fix details in <https://github.com/Kaskad-Lending/kaskad-squashed/pull/12>

Issue H-8: Recycled amount is not tracked accurately, partial amount will be stuck in the contract. [RE-SOLVED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/38>

Summary

`_autoRecycleEpoch` is triggered by the first claim after an epoch ends, but runs **before** users' epochs are closed via `closeUserEpochs`. This causes undistributed rewards recorded after recycling to be permanently stuck in the contract, never burned and never forwarded to the next epoch.

Vulnerability Detail

When a user claims rewards, the flow in `KaskadRewardsController.claimRewards()` executes in this order:

```
function claimRewards(...) external override whenNotPaused returns (uint256) {
    if (to == address(0)) revert InvalidToAddress();
    _maybeDistributePreviousEpoch(); // 1. Recycles previous epoch
    if (address(activityTracker) != address(0)) {
        activityTracker.closeUserEpochs(assets, msg.sender); // 2. Closes user's
        ↪ past epochs
    }
    return _claimRewards(assets, amount, msg.sender, msg.sender, to, reward);
}
```

Step 1 calls `_maybeDistributePreviousEpoch()` which calls `_autoRecycleEpoch(prevEpoch)`:

```
function _autoRecycleEpoch(uint256 epoch) internal {
    address reward = address(KSKD_TOKEN);

    // Skip if already recycled or nothing to recycle
    if (epochRecycled[epoch][reward]) return; // @audit returns on all subsequent
    ↪ calls

    uint256 undistributed = undistributedPerEpoch[epoch][reward];
    if (undistributed == 0) {
        epochRecycled[epoch][reward] = true;
        return;
    }

    epochRecycled[epoch][reward] = true; // @audit locks recycling permanently
}
```

```

uint16 toNextEpochBps = STRATEGY.undistributedToNextEpochBps();
uint256 toNextEpoch = (undistributed * toNextEpochBps) / 10000;
uint256 toBurn = undistributed - toNextEpoch;

if (toNextEpoch > 0) {
    recycledForEpoch[epoch + 1][reward] += toNextEpoch;
}
if (toBurn > 0) {
    KSKD_TOKEN.burn(toBurn);
}
// ...
}

```

Step 2 then calls `closeUserEpochs`, which processes the claiming user's past epochs. For ineligible users, this calls `recordUndistributed()` – adding to `undistributedPerEpoch[prevEpoch]` **after** recycling has already completed and locked the epoch.

The issue is compounded by the fact that `closeUserEpochs` is only triggered per-user when they personally claim or interact with the protocol. There is no keeper or batch mechanism to close all users' epochs before recycling occurs. This means:

1. **First claimer** in epoch N triggers `_autoRecycleEpoch(N-1)` which reads whatever partial `undistributedPerEpoch[N-1]` exists and sets `epochRecycled = true`. Then their own `closeUserEpochs` may add more undistributed to epoch N-1.
2. **All subsequent claimers** hit the `if (epochRecycled[epoch][reward])` return guard at line 147 and skip recycling. Their `closeUserEpochs` continues adding to `undistributedPerEpoch[N-1]`.
3. The permissionless `recycleUndistributed()` cannot help – it reverts with `EpochAlreadyRecycled` since the flag is already set.

All undistributed amounts recorded after the first claim's auto-recycle are trapped in the contract.

Impact

- **KSKD tokens permanently stuck:** Undistributed rewards that should have been burned or recycled to the next epoch remain locked in the contract.
- **Understated recycledForEpoch:** The next epoch receives fewer recycled tokens than intended, reducing total rewards available for distribution.
- **Magnitude scales with user inactivity:** The more users that claim lazily (i.e., don't interact until well after the epoch ends), the larger the amount of tokens trapped. In a protocol where many users may only claim periodically, this could represent a significant portion of undistributed rewards each epoch.

Recommendation

We need a re-design of how the recycling mechanism works. Simplest would be to pool recycled amounts into the vault whenever they are generated and then distribute them in the next epoch. This means that undistributed tokens from epoch X could be given to $X + N$ and not necessarily $N+1$.

Discussion

KaiReytsu

Fixed. Fix details in <https://github.com/Kaskad-Lending/kaskad-squashed/pull/8>

Issue H-9: Attack can inflate rewards received due to incorrect balance tracking [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/39>

Summary

Reward emissions are calculated using the user's current scaled balance rather than a time-weighted balance, allowing an eligible user to deposit a large amount just before an epoch ends and receive inflated rewards for the entire epoch.

Vulnerability Detail

`supplierEligibleSince` is only set once, when a user first crosses the `supplierDepositMinUsd` threshold and is never updated on subsequent deposits:

<https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/blob/main/kaskad-squashed/src/dao/KaskadStrategy.sol#L521-L525>

```
// Only SET eligibility on entry, never reset here
// Eligibility reset is handled by ActivityTracker after rewards finalized
if (state.supplierEligibleSince == 0 && isEligible) {
    state.supplierEligibleSince = uint64(block.timestamp);
}
```

When epochs are sealed, rewards are computed using the user's **current** `scaledBalanceOf` not a time-weighted average:

<https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/blob/main/kaskad-squashed/src/dao/KaskadActivityTracker.sol#L246>

```
uint256 userBalance = IScaledBalanceToken(asset).scaledBalanceOf(user);
```

<https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/blob/main/kaskad-squashed/src/dao/KaskadActivityTracker.sol#L308-L309>

```
uint256 epochRewards = (ctx.userBalance *
    (endIndex - startIndex)) / ctx.assetUnit;
```

While `handleAction` seals **past** epochs with the pre-transfer (old) balance, the **current** epoch remains open and will be sealed later with the new inflated balance:

<https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/blob/main/kaskad-squashed/src/dao/KaskadActivityTracker.sol#L125-L129>

```
// 2.5 Seal past epochs using pre-transfer balance (flash loan protection)
//     handleAction is called BEFORE balance update, so scaledBalanceOf
```

```
//      returns the OLD balance. This locks in correct rewards before
//      the balance changes.
_closeForAsset(user, asset, currentEpoch);
```

`_closeForAsset` only processes epochs where `processedEpoch < ctx.currentEpoch`, so the current epoch is never sealed at deposit time.

Attack scenario:

1. Day 1 of epoch: User supplies the minimum (\$100) to set `supplierEligibleSince`.
2. Day 1–29: User maintains the minimum deposit, accumulating uptime past the 90% supplier threshold (27 of 30 days).
3. Day 29: User deposits a much larger amount (e.g. \$1M). `handleAction` seals all past epochs with the old balance, but the current epoch remains open.
4. Next epoch: When the current epoch is eventually sealed (via `handleAction` or `closeUserEpochs`), `scaledBalanceOf` returns the new large balance, and rewards for the entire previous epoch are calculated as `$1M_balance * indexGrowth` instead of `$100_balance * indexGrowth`.

This is the most capital-efficient execution the attacker only locks minimal capital for ~30 days and a large amount just to front-run the next epoch boundary update.

Impact

- Users can receive disproportionately inflated KSKD emission rewards relative to their actual contribution over the epoch.
- Honest long-term suppliers who maintain large positions for the full epoch are diluted, as their share of the fixed epoch emissions is reduced by the attacker's inflated claim.
- The attack is repeatable every epoch with minimal capital lockup

Recommendation

Use time-weighted balances for reward calculation. For example, track cumulative `balance * time` per user per epoch and divide by epoch duration when computing rewards.

Discussion

KaiReytsu

Fixed. Fix details in <https://github.com/Kaskad-Lending/kaskad-squashed/pull/10>

DemoreXTess

Twab causes insolvency and distribute less rewards because index is still calculated by total supply, system should calculate it accurately, sum of all the twabs should be equal to divisor value at index calculation.

```
uint256 actualTotalSupply = IScaledBalanceToken(asset)
    .scaledTotalSupply();

uint256 previousEpochEnd = EpochConfig.end(currentEpoch - 1);

uint256 index = REWARDS.updateRewardIndexForTracker(
    asset,
    reward,
    actualTotalSupply,
    previousEpochEnd // + epoch-boundary clamping
);
```

KaiReytsu

I've made the updates, you can review them here:

<https://github.com/Kaskad-Lending/kaskad-squashed/pull/13>

Issue M-1: Contracts can be initialized by anyone [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/21>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The multiple contracts contain an unprotected `initialize()` function that can be front-run by malicious actors during deployment, potentially causing a Denial of Service (DoS).

Vulnerability Detail

The `initialize()` function on line 68 lacks access control restrictions. While it uses OpenZeppelin's initializer modifier to prevent re-initialization, anyone can call this function after contract deployment:

```
function initialize(address growthPool) external initializer {
```

The function is intended to be called by the deployer to set the `GROWTH_POOL_ROLE`, but since it's publicly callable, an attacker monitoring the mempool can front-run the legitimate initialization transaction.

Impact

An attacker can front-run the initialization with a zero address or invalid address, causing the contract to be permanently misconfigured. Since the initializer modifier prevents re-initialization, the contract would need to be redeployed.

Tool Used

Manual Review

Recommendation

Add access control to the `initialize()` function by restricting it to only be callable by an admin role or the deployer.

Discussion

KaiReytsu

Acknowledged. All `initialize()` functions use the OpenZeppelin `initializer` modifier, guaranteeing single-use execution. If front-run, the deployer's call reverts, the attack is immediately detected and the contract is redeployed. At initialization time the contracts are not yet wired into the protocol, so no funds or state are at risk.

Issue M-2: Incomplete Budget Distribution Design [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/22>

Summary

The GrowthPool contract's release schedule mathematically cannot distribute 100% of the totalBudget, leaving approximately 10.9% of funds permanently locked in the contract.

Vulnerability Detail

The release rate constants sum to only 89.1% of the total budget:

1. TGE Release: 5.7%
2. Monthly Releases: $17 \text{ months} \times 4\% = 68\%$
3. Small Milestones (levels 1-3): $3 \times 1.4\% = 4.2\%$
4. Large Milestones (levels 4-7): $4 \times 2.8\% = 11.2\%$
5. Total: 89.1%

There is no mechanism to release or recover the remaining 10.9% of the budget.

Impact

Approximately 10.9% of the totalBudget will remain permanently locked in the contract with no way to distribute or recover it.

Tool Used

Manual Review

Recommendation

Adjust the release rate constants to sum to 100%, add a final release mechanism that distributes remaining funds after all milestones and monthly releases are complete

Discussion

KaiReytsu

Fixed. Fix details in <https://github.com/Kaskad-Lending/kaskad-squashed/pull/2>

Issue M-3: Reward Rate Not Updated on Distribution [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/23>

Summary

The GrowthPool contract distributes tokens to KaskadRewardsController via direct transfers, but this mechanism fails to update the reward emission rate in the controller, causing distributed tokens to become inaccessible or improperly distributed to users.

Vulnerability Detail

When `triggerMonthlyRelease()`, `triggerTGE()`, or `triggerMilestone()` are called, the `_distribute()` function sends tokens to the rewardsController via `safeTransfer`:

```
if (toTvl > 0) {  
    kskd.safeTransfer(rewardsController, toTvl);  
}
```

However, simply transferring tokens to the KaskadRewardsController does not automatically update the reward rate per second or notify the controller of new available rewards. The controller likely maintains internal accounting of emissions and reward rates that must be explicitly updated when new funds arrive.

Impact

Tokens transferred to the rewardsController may sit idle without being distributed to TVL participants, effectively locking protocol funds.

Tool Used

Manual Review

Recommendation

Implement a callback mechanism or dedicated function on the KaskadRewardsController that the GrowthPool calls after transferring tokens. This function should update the internal reward rate accounting to reflect the newly available funds for distribution. Consider using a pattern like `notifyRewardAmount()` commonly seen in staking reward contracts.

Discussion

KaiReytsu

Fixed. Fix details in <https://github.com/Kaskad-Lending/kaskad-squashed/pull/3>

Issue M-4: Missing Quorum Requirement Allows Single-Voter Decision Control [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/24>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The KaskadGovernor contract lacks a minimum quorum requirement for governance decisions, allowing a single voter to unilaterally determine protocol parameters by voting at the last moment of a voting window when no other votes have been cast.

Vulnerability Detail

The `vote()` function and `finalizeEpoch()` function do not implement any quorum checks. When finalizing an epoch, if `totalWeight` for a decision is zero (no votes cast), the system defaults to `p.minBps`:

```
uint256 totalWeight = sumWeights[epoch][d];
uint16 value = p.minBps;
if (totalWeight > 0) {
    value = uint16(sumWeightedValues[epoch][d] / totalWeight);
}
```

This means any single voter regardless of how small their voting weight—can determine a decision's outcome if they are the only participant.

Impact

An attacker can monitor voting activity and strategically vote at the end of the voting window for decisions with zero participation, setting critical protocol parameters to their preferred values.

Tool Used

Manual Review

Recommendation

Implement a minimum quorum requirement that must be met for a vote to be considered valid. If the quorum is not reached, either use the previous epoch's values, apply a default safe value, or extend the voting period. Consider setting the quorum as a

percentage of total voting power in the system to ensure meaningful participation before critical parameter changes are enacted.

Discussion

KaiReytsu

Acknowledged. The absence of a quorum is an intentional design choice. A voter cannot set an arbitrary value, `vote()` reverts if `valueBps` falls outside `[minBps, maxBps]`. The ranges are hardcoded. Even a single voter with minimal weight cannot exceed these bounds. When no votes are cast, `minBps` applies as the most conservative default.

Issue M-5: Emissions are equal for all assets independent of their TVL [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/32>

Summary

`EmissionManager.configureAssets()` distributes KSKD reward emissions equally across all configured assets regardless of their TVL, allowing rational actors to concentrate deposits in low-TVL pools to capture disproportionately high reward APY.

Vulnerability Detail

In `EmissionManager.configureAssets()`, the total epoch emission is divided equally among all assets:

<https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/blob/main/kaskad-squashed/src/dao/EmissionManager.sol#L77-L79>

```
uint88 emissionPerSecond = uint88(
    epochEmission / DURATION / assets.length
);
```

This same `emissionPerSecond` is then assigned to every asset in the `config loop`. The Aave RewardsController distributes rewards proportionally to each user's share of an asset's total supply. Because every pool receives identical emissions, a pool with \$100K TVL emits the same KSKD per second as a pool with \$100M TVL – meaning depositors in the small pool earn ~1000x the reward APY.

Impact

- Protocol emission budget is spent inefficiently, rewarding low-TVL pools that contribute little to overall protocol health.
- Rational users are incentivized to migrate liquidity away from core lending markets into low-TVL pools, degrading the protocol's primary utility.

Recommendation

Weight `emissionPerSecond` per asset based on TVL or governance-set allocation weights rather than dividing equally.

Alternatively, introduce a gauge-style weighting system (similar to Curve) where governance or token holders vote on per-asset emission allocation each epoch.

Discussion

KaiReytsu

Fixed. Fix details in <https://github.com/Kaskad-Lending/kaskad-squashed/pull/6>

DemoreXTess

Rates will be updated two times for growth pool, growth pool already increase the rate of reward distribution while `notifyGrowthPoolReward` call and it increases again for same rewards while `updateEmissionWithRollover` call

KaiReytsu

Made the updates. PR: <https://github.com/Kaskad-Lending/kaskad-squashed/pull/15>

Issue M-6: Un-stake penalty can be circumvented with sybil accounts [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/33>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

StKSKDVault's partial unstake time penalty can be fully bypassed by splitting a stake across multiple accounts (Sybil), allowing users to unstake portions without any entryTime penalty on remaining funds.

Vulnerability Detail

When a user partially unstakes from StKSKDVault, the contract penalizes the remaining balance by shifting entryTime forward proportionally to the fraction withdrawn:

<https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/blob/main/kaskad-squashed/src/pools/StKSKDVault.sol#L74-L80>

```
} else {  
    // Partial unstake - shift entry time forward (lose accumulated time)  
    uint64 oldEntry = entryTime[msg.sender];  
    uint64 elapsed = uint64(block.timestamp) - oldEntry;  
    uint64 timeLost = uint64((uint256(elapsed) * shares) / oldShares);  
    entryTime[msg.sender] = oldEntry + timeLost;  
}
```

This mechanism is designed so that withdrawing a portion of staked tokens reduces the holding duration (and therefore voting power) on the remaining tokens. However, this penalty is applied per-address and can be completely avoided by distributing tokens across multiple accounts before staking.

Example scenario:

A user wants to stake 1000 KSKD but retain the ability to partially exit without penalty:

1. User splits 1000 KSKD across 10 accounts (100 KSKD each) and stakes on all 10.
2. After accumulating time D, the user wants to withdraw 10% (100 KSKD).
3. They fully unstake from one account – entryTime resets to 0 on that account only.
4. The other 9 accounts retain holdingDuration = D with zero penalty.

The penalty is entirely circumvented. This advantage compounds with repeated partial withdrawals – a Sybil user can unstake incrementally while always preserving 100% of accumulated time on remaining funds.

Impact

- The partial unstake time penalty in `StKSKDVault` becomes ineffective as any user can trivially avoid it by using multiple addresses.
- Users who split across accounts retain full voting power on remaining stakes, gaining an unfair governance advantage over honest single-account stakers who suffer the intended penalty.

Recommendation

This is hard to mitigate since we can not track sybil accounts. The simplest solution is to remove the penalty when unstaking so that normal users also retain their voting power.

Discussion

KaiReytsu

Acknowledged. Sybil attacks (splitting stakes across multiple wallets) are a fundamental limitation of pseudonymous on-chain accounts that cannot be solved without making `stKSKD` non-transferable, which would break composability. The penalty mechanism uses a weighted-average formula that correctly reduces accumulated time proportionally on unstake. The penalty increases the cost of single-wallet manipulation, and any interaction within a single account is correctly pro-rated.

Issue M-7: User could receive rewards they are not entitled to if no activity happen during an epoch [RESOLVED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/40>

Summary

Coalesced snapshots in `KaskadActivityTracker._snapshotEpoch()` bypass per-epoch eligibility checks, allowing users who were ineligible for a given epoch to receive that epoch's full rewards by having them bundled under a later epoch's eligibility check.

Vulnerability Detail

When no interaction occurs on an asset for an entire epoch, `_snapshotEpoch()` only records a snapshot for `currentEpoch - 1`, skipping all intermediate epochs:

```
uint256 targetEpoch = currentEpoch - 1;
epochIndexSnapshot[asset][reward][targetEpoch] = index;
```

This single snapshot accumulates the full index delta for every skipped epoch. When `_processEpochsForReward()` later processes the user's rewards, it calls `_findNextSnapshot()` which scans forward from `lastClaimedEpoch` and lands on the single coalesced snapshot. Eligibility is then checked only once, for that snapshot's epoch:

```
bool eligible = _isUserEligibleForEpoch(ctx, snapshotEpoch);
```

No eligibility check is performed for the skipped epochs whose emissions are bundled into this snapshot.

Scenario:

1. User deposits late in epoch 7, well below the uptime threshold for epoch 7 eligibility
2. `handleAction` fires: epoch 6 is snapshotted, `userIndex` is set to the epoch 6 boundary index, `lastClaimedEpoch` = 7
3. No interactions occur on this asset during the entirety of epoch 8
4. In epoch 9, another user interacts with the asset. `_snapshotEpoch` records only epoch 8 (`targetEpoch` = `9 - 1` = 8). Epoch 7 receives no snapshot
5. The first user's rewards are processed: `_findNextSnapshot(7, 9)` finds no snapshot at epoch 7, returns epoch 8
6. The full index delta (covering all of epoch 7 and epoch 8 emissions) is checked against epoch 8 eligibility only

7. The user was present for all of epoch 8, passes the uptime threshold for epoch 8, and receives the combined rewards, including the full epoch 7 emissions they were never eligible for

Impact

Users who are ineligible for a given epoch (due to insufficient uptime) receive that epoch's full rewards when those rewards get coalesced into a later epoch's snapshot. The leaked amount equals the entire epoch's emissions proportional to the user's balance, not merely a partial-epoch fraction.

Recommendation

When snapshotting after a gap, create snapshots for each skipped epoch individually so that `_processEpochsForReward` can check eligibility per epoch.

Discussion

KaiReytsu

Fixed. Fix details in <https://github.com/Kaskad-Lending/kaskad-squashed/pull/11>

Issue M-8: Following epoch will receive less emissions than expected when snapShot() is called late [RE-SOLVED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/41>

Summary

Late epoch snapshots in `KaskadActivityTracker._snapshotEpoch()` cause the following epoch to receive fewer rewards than intended, because `lastUpdateTimestamp` is advanced to `block.timestamp` rather than the previous epoch's end boundary.

Vulnerability Detail

When a user interacts with the protocol after an epoch boundary, `_maybeSnapshotPreviousEpoch()` triggers `_snapshotEpoch()` to record the global reward index for the most recently completed epoch:

```
uint256 index = REWARDS.updateRewardIndexForTracker(
    asset,
    reward,
    actualTotalSupply
);

uint256 targetEpoch = currentEpoch - 1;
epochIndexSnapshot[asset][reward][targetEpoch] = index;
```

`updateRewardIndexForTracker()` calls Aave's `_updateRewardData()`, which updates the index via `_getAssetIndex()`:

```
uint256 currentTimestamp = block.timestamp > distributionEnd
    ? distributionEnd
    : block.timestamp;
uint256 timeDelta = currentTimestamp - lastUpdateTimestamp;
uint256 firstTerm = emissionPerSecond * timeDelta * assetUnit;
```

After computing the new index, `_updateRewardData()` sets:

```
rewardData.lastUpdateTimestamp = block.timestamp.toUint32();
```

This is the root cause. `lastUpdateTimestamp` is advanced to `block.timestamp` which is already inside the current epoch. When the current epoch is later snapshotted, its index delta is computed from this advanced `lastUpdateTimestamp`, not from the actual epoch

start. The current epoch permanently loses the emissions for the time window between its start and the moment the late snapshot occurred.

Impact

An epoch receives less rewards than expected because the index calculation uses an incorrect start time, skipping the period between the epoch's actual start and the delayed snapshot of the prior epoch.

Recommendation

Enforce epoch-boundary-aware index updates. When snapshotting a completed epoch, clamp `lastUpdateTimestamp` to that epoch's end timestamp so that the index delta for each epoch strictly covers only the time within its boundaries.

Discussion

KaiReytsu

Fixed. Fix details in <https://github.com/Kaskad-Lending/kaskad-squashed/pull/5>

Issue L-1: No Vote Updating Mechanism Penalizes Active Participants [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/25>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The KaskadGovernor contract prevents users from updating their vote even when their voting power increases during the voting window, creating an unfair disadvantage for early voters and incentivizing last-minute voting behavior.

Vulnerability Detail

The contract tracks whether a user has voted via `hasVoted[epoch][msg.sender][decision]` and reverts if they attempt to vote again:

```
if (hasVoted[epoch][msg.sender][decision]) {  
    revert AlreadyVoted();  
}
```

While the contract does sync voting weight before recording a vote via `strategy.syncForVoting(msg.sender)`, users who vote early in the voting window cannot update their vote later if:

1. They acquire more KSKD tokens
2. Their voting weight increases due to position changes
3. They want to change their voted value based on new information

Impact

Users are incentivized to wait until the last moment of the voting window to maximize their voting power, reducing overall governance participation and engagement.

Tool Used

Manual Review

Recommendation

Implement a vote updating mechanism that allows users to change their vote within the voting window.

Discussion

KaiReytsu

Acknowledged. The contract uses an accumulator structure (`sumWeights`, `sumWeightedValues`) and does not store individual vote values, so subtracting an old vote for re-voting is not possible without a significant architecture change. Allowing re-votes would also open a manipulation vector: a user votes with high weight, withdraws their position, and re-votes with a different value while the old weight remains in the accumulator. `syncForVoting()` is called immediately before recording the vote, ensuring the weight reflects the user's actual positions at voting time. Combined with bounded decision ranges (`minBps/maxBps`), the impact of any individual vote is constrained regardless of timing.

Issue L-2: Missing Emission Manager Hook Before Reward Processing [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/29>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

The `handleAction()` function processes rewards before triggering new KSKD emissions from the `EmissionManager`, potentially causing users to miss rewards from the current epoch's emission distribution.

Vulnerability Detail

The function proceeds to update TVL accumulators, initialize user indices, snapshot epochs, and close past epochs without first ensuring that the `EmissionManager` has distributed the current epoch's KSKD rewards

Impact

Users may not see their full entitled rewards until a subsequent action triggers the emission distribution.

Tool Used

Manual Review

Recommendation

Call `EmissionManager.maybeDistributeEpoch()` at the beginning of `handleAction()` before any reward index operations.

Discussion

KaiReytsu

Acknowledged. `_maybeDistributePreviousEpoch()` is called at the start of every claim path (`claimRewards`, `claimRewardsToSelf`, `claimAllRewards`, `claimAllRewardsToSelf`). `handleAction()` does not distribute rewards to users, it only updates indices, snapshots, and TWAB accumulators. The actual reward calculation happens during claim, where emission is guaranteed to be distributed before computation begins. Additionally, Kaska

`dGovernor._executeEpochHooks()` calls `maybeDistributeEpoch` during epoch finalization, providing a second independent trigger point.

Issue L-3: Unbounded loop could revert due to exceeding block gas limit during reward distribution [ACKNOWLEDGED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/30>

This issue has been acknowledged by the team but won't be fixed at this time.

Summary

`_executeEpochHooks()` in `KaskadGovernor` performs a bare external call without `try/catch`, contradicting the inline comment and risking finalization DoS if the `emissionManager` reverts.

Vulnerability Detail

The `finalizeEpoch()` function calls `_executeEpochHooks()` at the end of finalization. The comment on [L199](#) states:

"safe, uses `try/catch` to never block finalization"

However, the implementation uses a bare external call with no `try/catch`:

```
function _executeEpochHooks(uint256 epoch) internal {
    if (emissionManager != address(0)) {
        IEmissionManager(emissionManager).maybeDistributeEpoch(epoch); // @audit no
        ↪ try/catch
    }
}
```

If `maybeDistributeEpoch()` reverts for any reason (e.g. out of gas, logic error, paused contract), the revert bubbles up and the entire `finalizeEpoch()` transaction fails. Since `epochFinalized[epoch]` is only set to `true` on [L195](#) before the hook call, the state is reverted and finalization can be retried – but it will keep failing as long as the hook reverts. The admin can recover by calling `setEmissionManager(address(0))` to disable the hook, but this introduces an unnecessary admin dependency for a critical governance function.

Recommendation

Wrap the external call in a `try/catch` as the comment states.

Discussion

KaiReytsu

Acknowledged. `createVesting()` and `createVestingBatch()` are gated by `ADMIN_ROLE`, so the `vestingHolders` array grows only by admin decision. The Community Round has a fixed, known set of investors, well below the ~1,000 holder gas limit threshold. The risk does not arise under normal protocol operation.

Issue L-4: Missing try-catch in KaskadGovernor [RE-SOLVED]

Source: <https://github.com/sherlock-audit/2026-02-kaskad-feb-16th-2026/issues/31>

Summary

`_executeEpochHooks()` in `KaskadGovernor` performs a bare external call without try/catch, contradicting the inline comment and risking finalization DoS if the `emissionManager` reverts.

Vulnerability Detail

The `finalizeEpoch()` function calls `_executeEpochHooks()` at the end of finalization. The comment on [L199](#) states:

"safe, uses try/catch to never block finalization"

However, the implementation uses a bare external call with no try/catch:

```
function _executeEpochHooks(uint256 epoch) internal {
    if (emissionManager != address(0)) {
        IEmissionManager(emissionManager).maybeDistributeEpoch(epoch); // @audit no
        ↪ try/catch
    }
}
```

If `maybeDistributeEpoch()` reverts for any reason (e.g. out of gas, logic error, paused contract), the revert bubbles up and the entire `finalizeEpoch()` transaction fails. Since `epochFinalized[epoch]` is only set to `true` on [L195](#) before the hook call, the state is reverted and finalization can be retried – but it will keep failing as long as the hook reverts. The admin can recover by calling `setEmissionManager(address(0))` to disable the hook, but this introduces an unnecessary admin dependency for a critical governance function.

Recommendation

Wrap the external call in a try/catch as the comment states.

Discussion

KaiReytsu

Fixed. Fix details in <https://github.com/Kaskad-Lending/kaskad-squashed/pull/4>

Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.